



Zespół Szkół Elektrycznych w Gorzowie Wielkopolskim

DOKUMENTACJA

Inventory Master– Aplikacje mobilne

Jan Matysik 4TP 2025 / 2026



Spis treści

Testowane funkcjonalności.....	3
Logika przetwarzania kodów kreskowych (BarcodeController).....	3
Warstwa dostępu do danych (Klasy DAO: Item, ItemInstance, Inventory).....	3
Zarządzanie danymi w widokach list (Recycler Adapters).....	3
Walidacja reguł biznesowych (FieldValidator).....	3
Centralna obsługa odpowiedzi sieciowych (CallbackBuilder).....	4
Wtyczki użyte w sprawozdaniu.....	6
Wnioski.....	6



Testowane funkcjonalności

Logika przetwarzania kodów kreskowych (BarcodeController)

Moduł ten odpowiada za interpretację danych wejściowych pochodzących ze skanera. Wykonuje parsowanie surowego ciągu znaków w celu wyodrębnienia kodu artykułu oraz numeru seryjnego instancji, a następnie koordynuje proces weryfikacji tych danych w bazie i zarządza nawigacją do widoku szczegółów.

Przy użyciu biblioteki Mockito symulowane są różne scenariusze odpowiedzi z warstwy danych (DAO): od poprawnych wyników, przez brak dopasowania w bazie, aż po błędy formatu. Test weryfikuje poprawność wyzwalanych komunikatów Toast oraz sprawdza, czy w przypadku sukcesu poprawnie inicjowany jest obiekt Intent z załączonymi danymi modelu.

Warstwa dostępu do danych (Klasy DAO: Item, ItemInstance, Inventory)

Klasy te stanowią interfejs komunikacyjny między logiką biznesową a REST API. Odpowiadają za mapowanie metod lokalnych na konkretne żądania sieciowe (GET, POST, DELETE, PUT) oraz przekazywanie danych do API.

Testy weryfikują poprawność interakcji z serwisami API. Sprawdzane jest, czy wywołanie metody w DAO skutkuje uruchomieniem odpowiedniego endpointa z właściwymi parametrami (np. ID artykułu, obiekt żądania) oraz czy żądanie zostaje poprawnie zakolejkowane do wykonania.

Zarządzanie danymi w widokach list (Recycler Adapters)

Adaptery zarządzają logiką prezentacji danych w komponencie RecyclerView. Odpowiadają za dynamiczne doczytywanie kolejnych stron danych (paginację), aktualizację stanu wewnętrznego listy oraz powiadamianie widoku o konieczności odświeżenia danych.

Testy monitorują zachowanie adaptera bez konieczności uruchamiania pełnego środowiska graficznego. Weryfikowana jest poprawność inkrementacji liczników stron po dodaniu nowej porcji danych oraz sprawdzane jest, czy metody powiadamiające widok o zmianach (notifyItemRangeInserted) są wywoływane z poprawnymi indeksami.

Walidacja reguł biznesowych (FieldValidator)

Komponent zapewnia integralność danych wejściowych przed ich wysłaniem do serwera. Sprawdza zgodność z założonymi regułami biznesowymi, takimi jak dopuszczalny zakres



długości znaków, poprawność znaków specjalnych w kodach oraz brak pustych wartości w polach obowiązkowych.

Przeprowadzana jest seria testów jednostkowych JUnit, które poddają validator próbom z użyciem danych poprawnych i błędnych. Test sprawdza, czy funkcja zwraca oczekiwany stan logiczny dla każdego z przypadków.

Centralna obsługa odpowiedzi sieciowych (CallbackBuilder)

Mechanizm ten standaryzuje sposób wykonywania zapytań do API. Pozwala na zdefiniowanie jednego globalnego handlera błędów (np. błąd serwera, brak połączenia), odciążając poszczególne moduły aplikacji od ciągle powtarzanej implementacji obsługi wyjątków.

Test weryfikuje poprawność przekierowywania statusów odpowiedzi do handlera błędów. Poprzez manualne wywoływanie zdarzeń onResponse oraz onFailure na zbudowanym obiekcie callbacku, sprawdzane jest, czy system poprawnie identyfikuje błędy.

```
Terminal Local x + -
PS D:\School\Class_4\Aplikacje_Mobilne\InventoryMaster4TP\mobile> ./gradlew test
BUILD SUCCESSFUL in 784ms
46 actionable tasks: 46 up-to-date
PS D:\School\Class_4\Aplikacje_Mobilne\InventoryMaster4TP\mobile> 
```

Ilustracja 1: Wykonanie komendy `./gradlew test` i wyświetlenie raportu testów.